

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – ARCHITECTURE / MODELING (SECURE INFRASTRUCTURE)

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO4 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Design secure DevSecOps infrastructure by developing architecture diagrams, modelling deployment workflows, Identity and Access Management (IAM) relationships, and GitOps workflows.		
Student Name:	M.GOKUL	Reg. No.:	192565067
Section / Batch:	CSE -CS	Date:	22/08/2026

Instructions to Students

- Design all architecture diagrams, deployment workflows, IAM relationships, and GitOps workflows originally and clearly.
- Use DiagrammeR or Draw.io for architecture modelling and maintain consistent labels, trust boundaries, data flows, and security controls.
- Clearly represent IAM roles/permissions, Infrastructure as Code (IaC), GitOps flow, and DevSecOps security integration wherever applicable.
- Include editable source files for diagrams, IaC/configuration files, methodology documentation, and all supporting project artifacts.
- Submit the GitHub repository link and final PDF report through Google Classroom before the specified deadline; verify that the repository is accessible and complete.

Question Type	Focus Area	Questions	Marks
ARCHITECTURE / MODELING	Design and Application Based	Q1	Q1 – 100
GRAND TOTAL:		100 Marks	

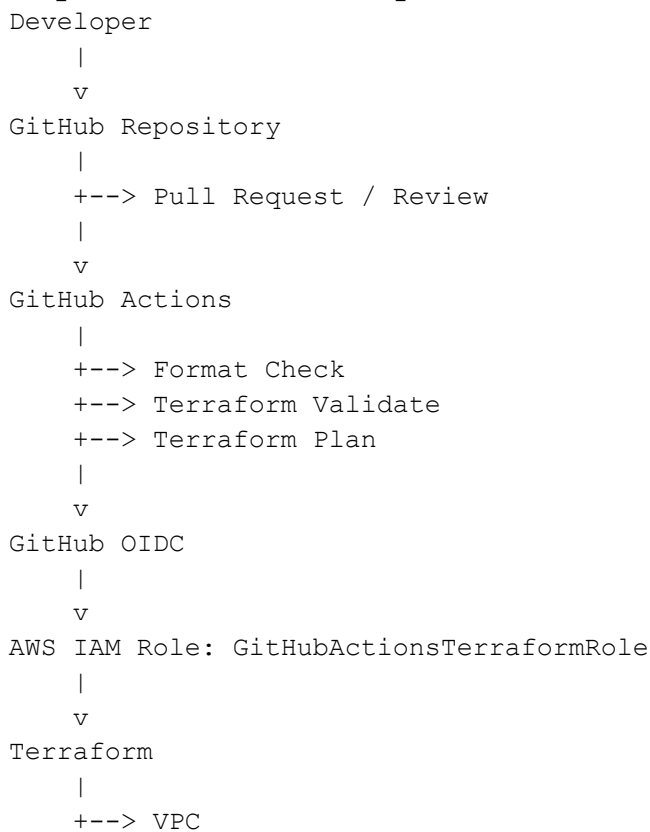
1. Aim

To design and model a secure DevSecOps-oriented AWS workflow that demonstrates Identity and Access Management (IAM), Infrastructure as Code (IaC) using Terraform, and a GitOps workflow using GitHub and GitHub Actions. The design follows the supplied student guide and the assessment rubric, with emphasis on least-privilege access, reusable infrastructure configuration, version control, automated validation, controlled deployment, and auditable workflow.

2. Objectives

1. Design an IAM model containing identities, roles, permissions, authentication and least-privilege controls.
2. Model AWS infrastructure provisioning through Terraform as Infrastructure as Code.
3. Store Terraform configuration and workflow files in GitHub for version control.
4. Implement a GitOps workflow using GitHub Actions for automated Terraform validation, planning and deployment.
5. Use GitHub OpenID Connect (OIDC) so that GitHub Actions can obtain temporary AWS credentials instead of storing permanent AWS access keys.
6. Integrate security validation, approval, logging and audit controls into the workflow.
7. Demonstrate a Git-based infrastructure change and its automated propagation to AWS.

3. Proposed AWS DevSecOps Architecture



```

+--> Public Subnet
+--> Internet Gateway
+--> Route Table
+--> Security Group
+--> EC2
|
v
AWS Infrastructure

```

```

Git change --> GitHub Actions --> Terraform --> AWS desired state

```

The supplied student guide defines the final workflow as Student → GitHub Repository → GitHub Actions → AWS IAM Role → Terraform → VPC → Subnet → Security Group → EC2. It also specifies that GitHub Actions authenticates to AWS using OIDC rather than storing a permanent AWS access key and secret key in GitHub. [filecite\turn1file0\L29-L41](#)

4. IAM Model and Access Control Design

The IAM design provides controlled access to AWS resources and follows the principle of least privilege. The student guide specifically requires observation of IAM Users, User Groups, Roles and Policies, and explains that an IAM policy defines permissions while an IAM role provides temporary permissions to an authorized principal. [filecite\turn1file0\L55-L58](#)

Identity / Component	Role	Required Access	Security Control
Developer	Application / IaC developer	Repository access and Pull Request creation	No direct production deployment
GitHub Actions	CI/CD service identity	Assume approved AWS deployment role	OIDC; temporary credentials
GitHubActionsTerraformRole	AWS deployment role	Only faculty/lab-approved Terraform permissions	Restricted trust policy
Reviewer	Change reviewer	Review and approve Pull Requests	Protected main branch
AWS Administrator / Lab Admin	Administrative support	Only required administrative operations	Separate privileged access

5. IAM Authentication and Authorization

- GitHub Actions authenticates to AWS using OpenID Connect (OIDC).
- The AWS IAM role is configured as a Web Identity role.
- The role trust relationship identifies the GitHub OIDC provider.
- The trust policy restricts the repository and main branch.

- Only permissions supplied or approved by the faculty or AWS lab administrator should be attached.
- AdministratorAccess should not be attached simply to make the workflow operate.
- Temporary credentials are preferred over permanent access keys.
- IAM permissions should follow least privilege.

6. OIDC Trust Relationship

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated":
          "arn:aws:iam::YOUR_ACCOUNT_ID:oidc-
provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
          "token.actions.githubusercontent.com:sub":
            "repo:YOUR-USERNAME/devsecops-aws-lab:ref:refs/heads/main"
        }
      }
    }
  ]
}
```

The supplied guide gives this trust-policy structure and instructs the student to replace the account ID and GitHub username. It also states that if the institution or GitHub configuration uses a different OIDC subject format, the exact required subject should be used rather than weakening the trust policy. [filecite](#)[turn1file0](#)[L216-L243](#)

7. Infrastructure as Code (IaC) Model

Terraform is used to describe the AWS infrastructure as code. The supplied guide requires Terraform configuration for a VPC, subnet, Internet Gateway, route table, security group and EC2 instance. [filecite](#)[turn1file0](#)[L20-L28](#)

7.1 Terraform Project Structure

```
devsecops-aws-lab/
|-- main.tf
|-- variables.tf
|-- outputs.tf
|-- .gitignore
`-- .github/
```

```
`-- workflows/
   |-- terraform.yml
```

7.2 Terraform Resources

Resource	Purpose	Configuration
AWS Provider	Connect Terraform to AWS	Region supplied through aws_region variable
VPC	Isolated AWS network	CIDR 10.10.0.0/16
Internet Gateway	Internet connectivity	Attached to lab VPC
Public Subnet	Public network segment	CIDR 10.10.1.0/24
Route Table	Network routing	Default route through Internet Gateway
Security Group	EC2 traffic control	HTTP TCP port 80 ingress
EC2 Instance	Web workload	Amazon Linux AMI and t3.micro

8. Main Terraform Configuration

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_vpc" "lab" {
  cidr_block           = "10.10.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "devsecops-lab-vpc"
  }
}

resource "aws_internet_gateway" "lab" {
  vpc_id = aws_vpc.lab.id

  tags = {
    Name = "devsecops-lab-igw"
  }
}
```

```

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.lab.id
  cidr_block        = "10.10.1.0/24"
  map_public_ip_on_launch = true

  tags = {
    Name = "devsecops-lab-public-subnet"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.lab.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.lab.id
  }

  tags = {
    Name = "devsecops-lab-public-rt"
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "web" {
  name          = "devsecops-lab-web-sg"
  description   = "Allow HTTP for assignment demonstration"
  vpc_id        = aws_vpc.lab.id

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "devsecops-lab-web-sg"
  }
}

```

```

}

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name     = "name"
    values   = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name     = "virtualization-type"
    values   = ["hvm"]
  }
}

resource "aws_instance" "web" {
  ami                  = data.aws_ami.amazon_linux.id
  instance_type        = var.instance_type
  subnet_id            = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y httpd
    systemctl enable httpd
    systemctl start httpd
    echo "<h1>DevSecOps IAM + IaC + GitOps Assignment</h1>" >
    /var/www/html/index.html
  EOF

  tags = {
    Name = "devsecops-lab-ec2"
  }
}

```

9. Variables and Outputs

9.1 variables.tf

```

variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "ap-south-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}

```



```

    default      = "t3.micro"
}

```

9.2 outputs.tf

```

output "vpc_id" {
    value = aws_vpc.lab.id
}

output "instance_id" {
    value = aws_instance.web.id
}

output "public_ip" {
    value = aws_instance.web.public_ip
}

output "website_url" {
    value = "http://${aws_instance.web.public_ip}"
}

```

10. Version Control and GitHub

The Terraform configuration is stored in a GitHub repository named devsecops-aws-lab. The guide recommends using a private repository unless the faculty specifically requires a public repository. The repository contains the Terraform files and GitHub Actions workflow.

□ filecite □ turn1file0 □ L197-L208 □

```

git init
git add .
git commit -m "Initial Terraform infrastructure"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/devsecops-aws-lab.git
git push -u origin main

```

11. GitOps Workflow

GitOps treats Git as the source of the desired infrastructure configuration. A change is made in Git, GitHub Actions detects the change, Terraform validates and plans the configuration, and the approved workflow applies the desired state to AWS.

```

Infrastructure Change
    |
    v
Git Branch / Commit
    |
    v
Pull Request / Review
    |
    v
GitHub Actions

```

```

      |
      +--> Terraform Format Check
      +--> Terraform Validate
      +--> Terraform Plan
      |
      v
Push to main
      |
      v
OIDC Authentication
      |
      v
AWS IAM Role
      |
      v
Terraform Apply
      |
      v
AWS Infrastructure

```

The guide's GitOps demonstration changes the EC2 Name tag in main.tf, commits and pushes the change, then verifies that GitHub Actions detects the modification and Terraform updates the AWS instance. This demonstrates that the desired infrastructure was changed in Git and automation propagated the change to AWS. [filecite](#)[turn1file0](#)[L301-L312](#)

12. GitHub Actions Workflow

```

name: Terraform AWS Deployment

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  id-token: write
  contents: read

env:
  AWS_REGION: ap-south-1
  TF_IN_AUTOMATION: "true"

jobs:
  terraform:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

```

```

- name: Configure AWS credentials with OIDC
  uses: aws-actions/configure-aws-credentials@v5
  with:
    role-to-assume:
arn:aws:iam::YOUR_ACCOUNT_ID:role/GitHubActionsTerraformRole
    aws-region: ${ env.AWS_REGION }

- name: Setup Terraform
  uses: hashicorp/setup-terraform@v3

- name: Terraform Init
  run: terraform init

- name: Terraform Format Check
  run: terraform fmt -check -recursive

- name: Terraform Validate
  run: terraform validate

- name: Terraform Plan
  run: terraform plan -out=tfplan

- name: Terraform Apply
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  run: terraform apply -auto-approve tfplan

```

The supplied workflow uses OIDC permissions, checks out the repository, configures AWS credentials through the GitHub OIDC role, initializes Terraform, checks formatting, validates the configuration, generates a plan, and applies the plan only when the workflow is a push to main.

filecite turn1file0 L247-L283

13. Security Integration

Stage	Security / Control	Purpose
Identity	IAM + OIDC	Temporary authenticated access to AWS
Repository	Protected main branch / review	Prevent uncontrolled changes
Terraform	fmt and validate	Detect formatting and configuration errors
Plan	Terraform plan	Preview infrastructure changes
Deployment	Restricted IAM role	Limit AWS permissions
Audit	GitHub Actions history + AWS logs	Trace changes and executions
Secrets	.gitignore and no credentials in Git	Prevent sensitive information exposure
Cleanup	terraform destroy	Remove assignment resources and reduce charges

14. Secure Git Configuration

```
.terraform/  
*.tfstate  
*.tfstate.*  
.terraform.lock.hcl  
crash.log  
*.tfvars  
*.tfplan
```

The student guide explicitly warns not to upload AWS credentials, private keys, passwords or Terraform state files to GitHub. [filecite](#)[turn1file0](#)[L189-L200](#)

15. IaC Validation Workflow

```
terraform init  
terraform fmt -recursive  
terraform validate
```

The expected result is a successful Terraform validation. The guide also requires a screenshot of the successful validation output for submission. [filecite](#)[turn1file0](#)[L197-L200](#)

16. AWS Resource Verification

8. Verify the VPC named devsecops-lab-vpc and CIDR 10.10.0.0/16.
9. Verify the public subnet named devsecops-lab-public-subnet and CIDR 10.10.1.0/24.
10. Verify the security group devsecops-lab-web-sg and inbound TCP port 80.
11. Verify the EC2 instance devsecops-lab-ec2 is running.
12. Copy the EC2 Public IPv4 address.
13. Open the public IP in a browser.
14. Verify the web page displays DevSecOps IAM + IaC + GitOps Assignment.

These verification steps and expected results follow the supplied guide.

[filecite](#)[turn1file0](#)[L291-L298](#)

17. GitOps Change Demonstration

```
# Change the EC2 tag in main.tf  
tags = { Name = "devsecops-lab-ec2-updated" }  
  
git add main.tf  
git commit -m "Update EC2 tag through GitOps"  
git push
```

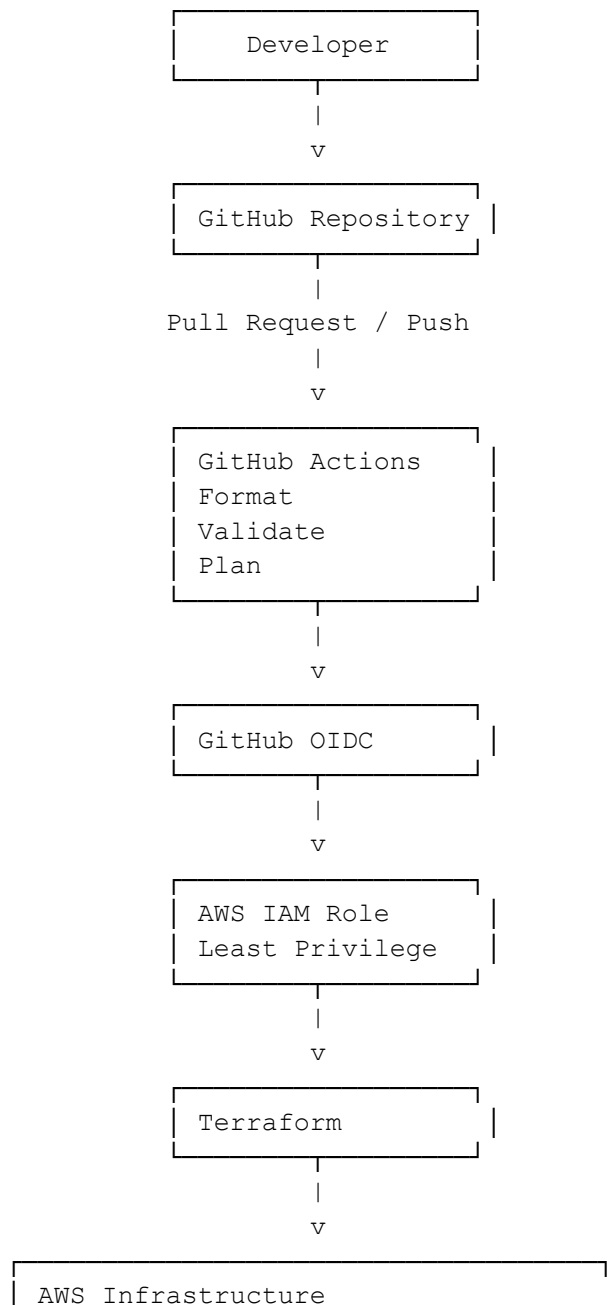
After the push, the GitHub Actions workflow is monitored. The Terraform Plan stage should show the tag modification, Terraform Apply updates AWS, and the EC2 Name tag is verified as devsecops-lab-ec2-updated. [filecite](#)[turn1file0](#)[L301-L312](#)

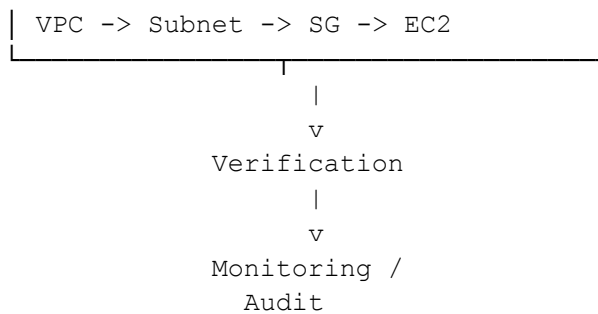
18. Pull Request Workflow

For a stronger DevSecOps workflow, infrastructure changes can be created on a separate branch and submitted as a Pull Request. The workflow validates and plans the Pull Request. After review and merge into main, the push to main triggers the Apply stage. [filecite](#)[turn1file0](#)[L313-L319](#)

```
git checkout -b infrastructure-change
# make a small Terraform change
git add .
git commit -m "Propose infrastructure change"
git push -u origin infrastructure-change
```

19. Complete Security and Automation Model





20. Documentation and Workflow Clarity

The architecture separates responsibilities between developers, GitHub, GitHub Actions, AWS IAM and Terraform. GitHub stores the desired configuration, GitHub Actions performs automated workflow stages, OIDC provides temporary AWS authentication, IAM controls permissions, and Terraform provisions the AWS resources. This separation supports traceability and reduces direct manual changes to infrastructure.

21. Required Evidence for Submission

- IAM GitHub Actions role and OIDC/trust configuration.
- Terraform files in the GitHub repository.
- Successful terraform validate output.
- Successful GitHub Actions workflow.
- Terraform Plan stage.
- AWS VPC.
- Public subnet.
- Security group showing port 80.
- Running EC2 instance.
- Browser showing the EC2 web page.
- Second GitHub Actions run for the GitOps change.
- Updated EC2 Name tag.
- Cleanup evidence if requested by faculty.

These evidence items are directly specified in the supplied student guide.

□ filecite □ turn1file0 □ L328-L331 □

22. Cleanup

After collecting the required evidence, the assignment resources must be removed. The guide states that cloud resources may incur charges and recommends destroying only resources belonging to the assignment. When running Terraform from an authenticated local environment, the cleanup command is `terraform destroy`, followed by careful review of the destruction plan.

□ filecite □ turn1file0 □ L319-L326 □

`terraform destroy`

23. Rubric Alignment

Assessment Criterion	Weight	Level-4 Evidence in this Answer
IAM Model and Access Control Design	20%	Identities, roles, permissions, OIDC, trust relationship and least-privilege controls are explicitly modelled.
Infrastructure as Code (IaC) Modelling	30%	Terraform project structure, AWS resources, variables, outputs, validation and automated plan/apply workflow are provided.
GitOps Workflow and Version Control	20%	GitHub repository, Pull Request, GitHub Actions, desired-state change and automated Terraform deployment are modelled.
Security Integration and Workflow Automation	20%	OIDC, IAM restrictions, validation, plan, controlled apply, protected workflow and credential-security practices are integrated.
Documentation and Workflow Clarity	10%	Architecture, IAM relationships, IaC, GitOps flow, verification, evidence and technical justification are documented.

24. Technical Justification

The proposed architecture is suitable for the assessment because it directly follows the supplied AWS student guide while addressing the assessment rubric. IAM controls access through a dedicated GitHub Actions role and OIDC trust relationship. Terraform models the AWS infrastructure as code and makes the configuration repeatable and version-controlled. GitHub provides version control and acts as the source of infrastructure changes. GitHub Actions automates validation, planning and deployment. The workflow therefore connects identity, infrastructure provisioning and GitOps into one auditable process.

25. Expected Final Result

The completed implementation demonstrates IAM-controlled CI/CD access, AWS infrastructure created through Terraform, Terraform configuration stored in GitHub, and an automated GitOps workflow through GitHub Actions. A change committed to Git is automatically validated, planned and applied to AWS. This matches the final result described in the supplied student guide.

□ filecite □ turn1file0 □ L362-L368 □

26. Conclusion

The designed DevSecOps infrastructure provides a controlled workflow for managing AWS infrastructure through IAM, Terraform and GitOps. The use of GitHub OIDC avoids storing permanent AWS access keys in the repository, while the restricted IAM role provides controlled AWS access. Terraform provides consistent Infrastructure as Code, and GitHub Actions automates validation, planning and deployment. Git-based changes provide traceability and support the desired-state model. Overall, the solution integrates IAM, IaC, GitOps, security and automation into a coherent secure infrastructure workflow aligned with the provided assessment requirements.

27. Viva Preparation – Short Answers

Question	Answer
What is IAM?	AWS Identity and Access Management is used to control authentication and authorization for AWS resources.
What is least privilege?	Giving an identity only the permissions required to perform its task.
IAM user vs IAM role?	A user represents an AWS identity, while a role provides permissions that an authorized principal can assume, commonly for temporary access.
What is OIDC?	OpenID Connect allows GitHub Actions to obtain temporary AWS credentials through a trusted identity relationship instead of storing permanent AWS access keys.
What is IaC?	Infrastructure as Code defines and manages infrastructure using configuration files rather than manual configuration.
What does terraform init do?	It initializes the Terraform working directory and downloads required provider components.
terraform plan vs apply?	Plan previews the proposed infrastructure changes; apply performs the approved changes.
What is Terraform state?	It records the relationship between Terraform configuration and managed infrastructure.
Why not commit credentials/state to GitHub?	They may contain sensitive credentials or infrastructure information and can create a serious security risk.
What is GitOps?	A workflow in which Git stores the desired configuration and changes are propagated through automation.
Why apply only after push to main?	It provides a controlled production path after changes have been reviewed and merged.
Why use Pull Requests?	They provide review, validation and controlled change management before merging infrastructure changes.
What is a VPC?	An isolated virtual network in AWS.

What is a security group?	A virtual firewall controlling network traffic for supported AWS resources such as EC2.
How did GitHub Actions obtain AWS permission?	Through GitHub OIDC and the AWS IAM GitHubActionsTerraformRole.

28. References

15. Student Guide – IAM, IaC and GitOps using AWS, Part 2 – 30 Marks, DEVSECOPS CO4 AT2. [filecite\turn1file0\L1-L12](#)
16. CO4 AT2 Assessment Tool 2 – Architecture / Modeling (Secure Infrastructure), SIMATS Engineering. [filecite\turn0file0\L2-L12](#)
17. CO4 Assessment Rubrics – IAM, IaC, GitOps, Security Integration and Documentation. [filecite\turn0file0\L44-L70](#)